

Лекция 13

Асинхронность и real-time web

Проектирование систем и продуктовая веб-разработка

Канев Антон Игоревич

Обучение и стажировки

Курсы VK <https://park.vk.company/curriculum/program/elective/>

- 2-3 семестровые программы (Android, iOS, Web, ML)
- Семестровые курсы VK (Android, iOS, Go,)

Курсы и стажировки выпускников ИУ5 и Технопарка из KTS
<https://metaclass.kts.studio/#courses>

- Бесплатные курсы по фронтенду (React/Next) и бэкенду (Asyncio)
- После курсов отбор на стажировки (Асинхронный Python, React)

Резюме

Советы по оформлению резюме <https://www.superjob.ru/pro/5320/>

- Опишите ваши проекты, приведите ссылки
- Не пишите общеизвестных вещей или просто список технологий
- Опишите ваши достижения, решения в проекте



Dan Abramov

gaearon

Working on @reactjs. Co-author of Redux and Create React App. Building tools for humans.

 @facebook

 [Sign in to view email](#)

 http://twitter.com/dan_abramov

Overview

Repositories 237

Projects 0

Stars 1.4k

Pinned

 [facebook/react](#)

A declarative, efficient, and flexible JavaScript library for building user interfaces.

 JavaScript ★ 140k 🍴 26.7k

 [facebook/create-react-app](#)

Set up a modern web app by running one command.

 JavaScript ★ 73.7k 🍴 17.3k

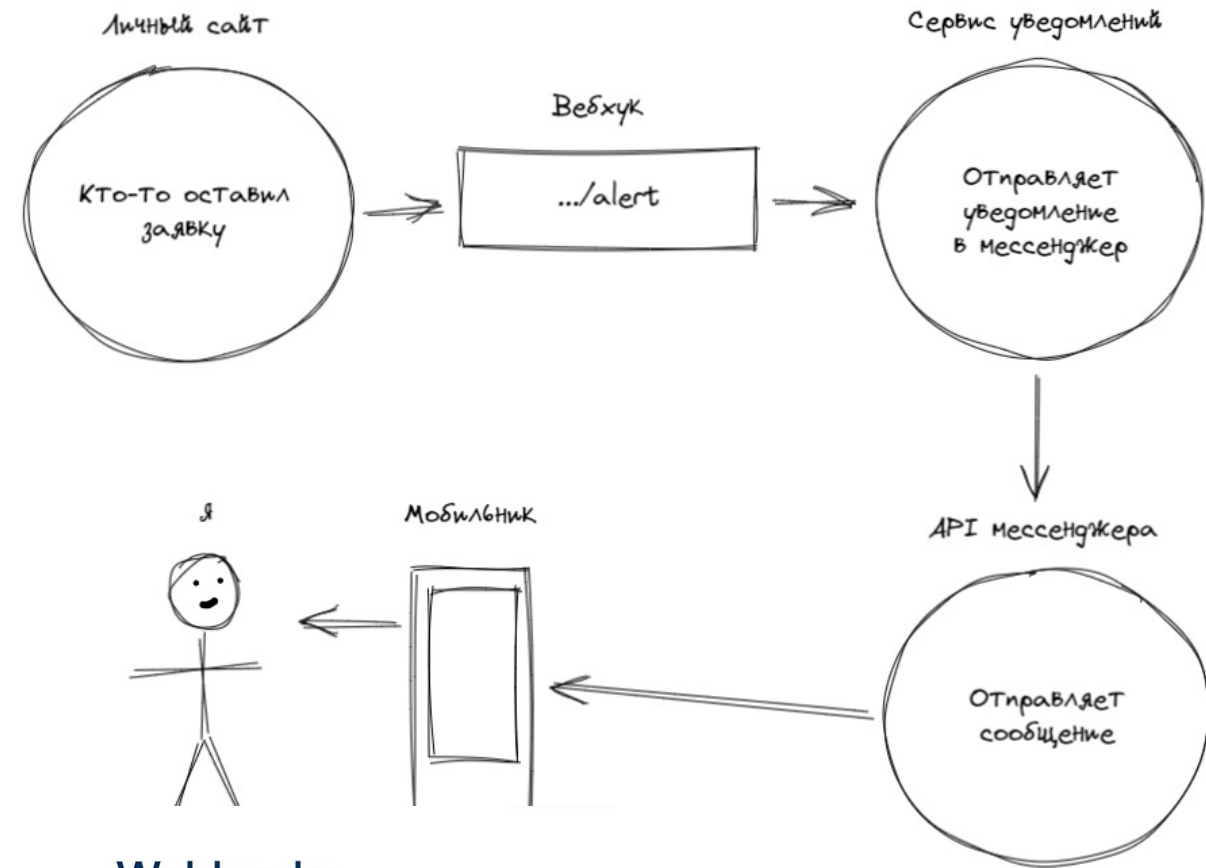
 [react-dnd/react-dnd](#)

Drag and Drop for React

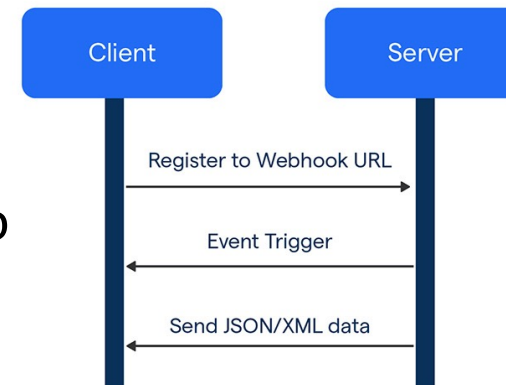
 TypeScript ★ 12.5k 🍴 1.4k

Webhook *

- Webhook – технология очень похожая на JSON-RPC между серверами для определенных видов запросов, которая появилась в 2007 году
- Смысл заключается в обратном вызове, когда вызываемый нами сервер должен вызвать наш метод или третий сервер
- Для Webhook используются отдельные библиотеки, дополнительные действия, и в курсе мы их не используем
- Вместо этого мы реализуем похожее простое взаимодействие, но с помощью обычного JSON-RPC

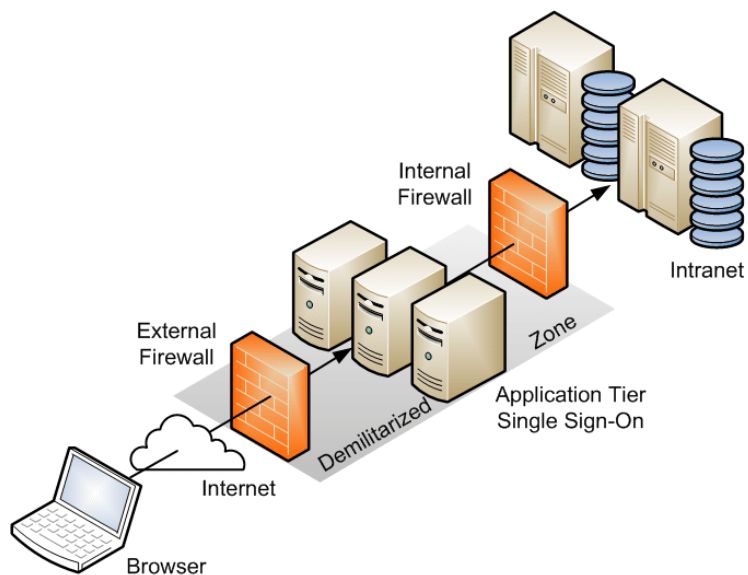


Webhooks

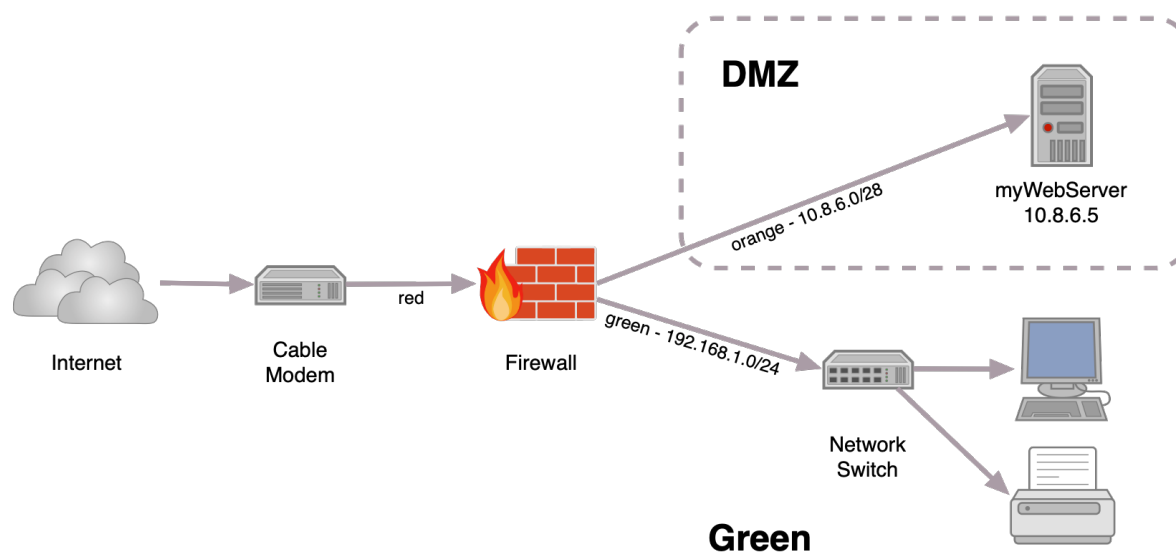
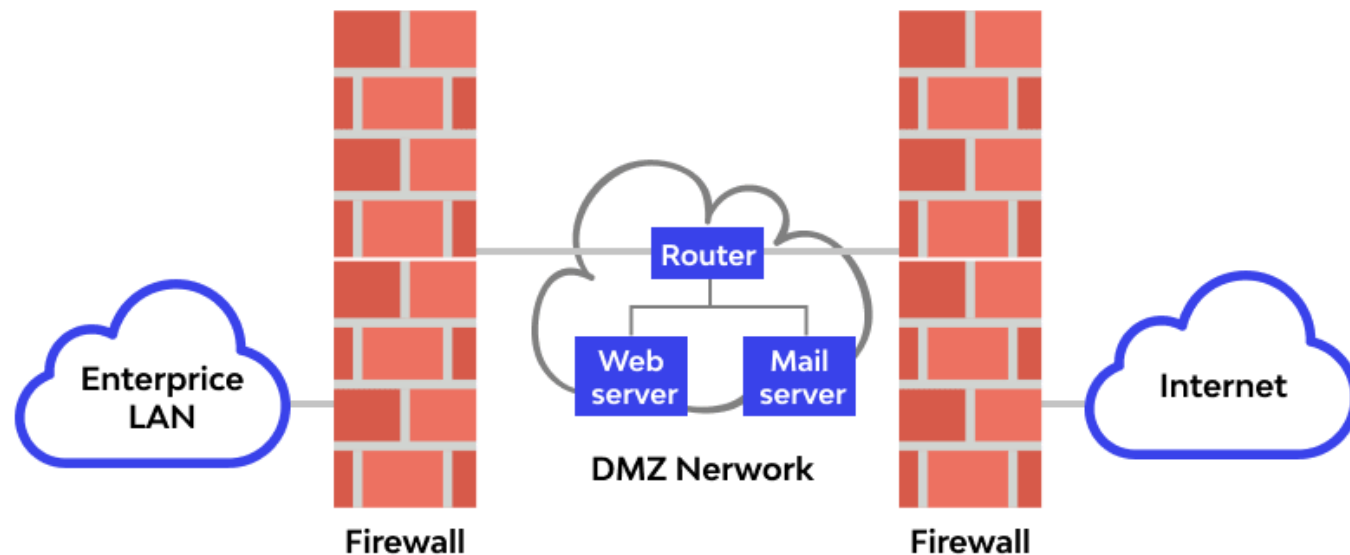


DMZ

- DMZ (Demilitarized Zone) – сегмент сети, содержащий общедоступные сервисы и отделяющий их от частных
- Разделение производим с помощью firewall

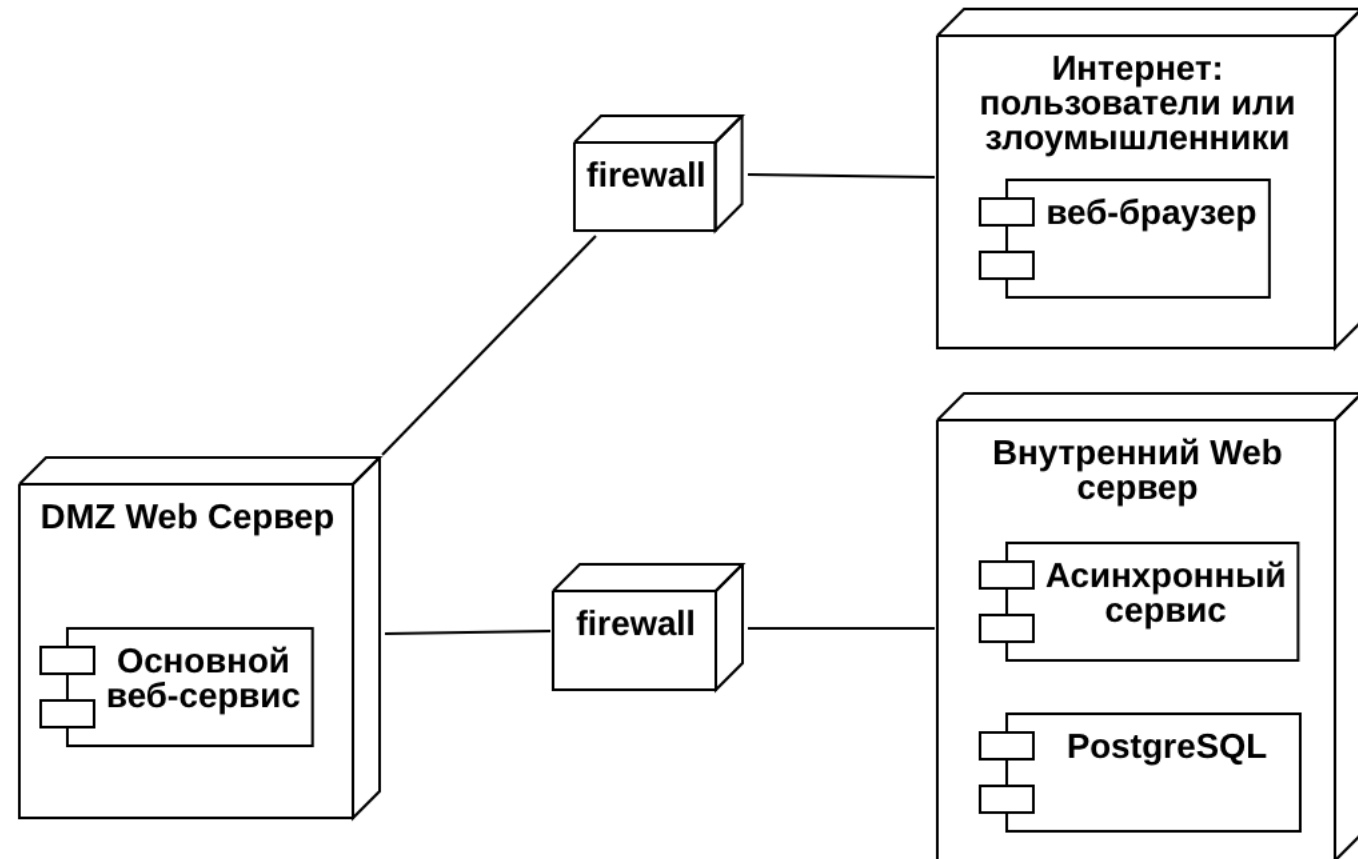


DMZ network architecture



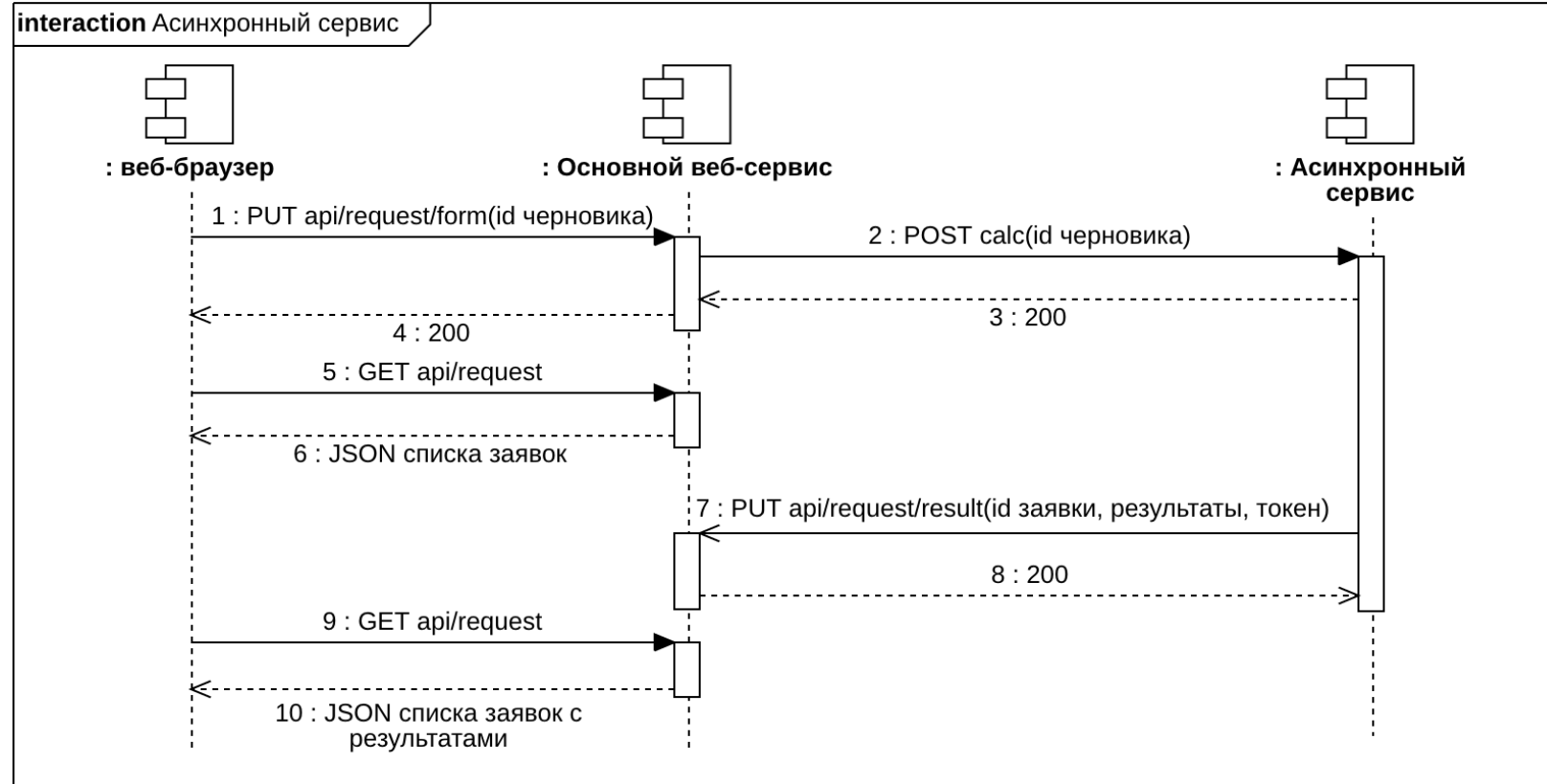
Как выглядит «DMZ» у нас? *

- У нас основной **веб-сервис** (скорее **реверс-прокси**) доступен извне – это хороший пример DMZ, остается только его оградить с помощью firewall
- Мы это не делаем, но подразумеваем, что эти сетевые настройки есть
- Таким образом все наши критичные **данные** (БД, Redis) и **асинхронный сервис** недоступны извне
- Но вот для обратного вызова от асинхронного к **основному** мы должны добавить какую-то **аутентификацию**



Sequence для асинхронного метода *

- При формировании заявки **1** мы обращаемся к асинхронному сервису **2** и сразу получаем ответ на фронтенде **4** – заявка сформирована
- Но запись результата в БД будет только через время отдельным вызовом **7**
- На фронтенде результат мы получим только когда сами спросим **9** об этом
- Запросы GET **5** и **9** аналогичны – это Polling из ДЗ
- На своей диаграмме вы **5** и **6** не указываете, мы это только подразумеваем



- Для псевдо аутентификации-авторизации используем **ключ-токен для ОСНОВНОГО сервиса в DMZ** в запросе **7**

Код лабораторной *

- Создаем новый асинхронный сервис, например на 8000 порте
- В нем будет один метод `/exchange_async`
- Данный метод будет запускать отложенное действие

```
@router.post("/exchange_async")
async def submit_request(data: dict, background_tasks: BackgroundTasks):
    if not data.get("request_id") or not data.get("exchange_rate") or not \
        raise HTTPException(status_code=400, detail="Invalid payload")

    background_tasks.add_task(
        send_exchange_result_async,
        request_id=data["request_id"],
        exchange_rate=data["exchange_rate"],
        bills=data["bills"]
    )

    return {"status": "ok"}
```


Код лабораторной *

- Наше отложенное действие заключается в конвертации валют
- После 5 секунд ожидания мы вызываем метод PUT /exchange_result нашего основного сервиса на 8080 или 3001 портах

```
class Settings:
    MAIN_SERVICE_URL = os.getenv("MAIN_SERVICE_URL", "http://localhost:3001")
    SECRET_TOKEN = os.getenv("SECRET_TOKEN", "MY_SECRET_TOKEN")

    @property
    def exchange_result_url(self) -> str:
        return f"{self.MAIN_SERVICE_URL}/api/exchange_result"
```

```
async def calculate_single_bill_async(bill: Dict, rate: float) -> Dict:
    await asyncio.sleep(random.uniform(1, 3))
    subtotal_rub = bill["denomination"] * bill["count"] * rate
    print(
        f"Рассчитана купюра {bill['denomination_id']}: {subtotal_rub:.2f} RUB")
    return {
        "denomination_id": bill["denomination_id"],
        "subtotal_rub": round(subtotal_rub, 2)
    }

async def calculate_bill_values_async(bills: List[Dict], rate: float) -> List[Dict]:
    tasks = [calculate_single_bill_async(bill, rate) for bill in bills]
    return await asyncio.gather(*tasks)
```

Просмотр результатов 2-ух сервисов

The image displays two screenshots of the Postman application, showing the results of two different API requests.

Left Screenshot (GET Request):

- Method:** GET
- URL:** http://0.0.0.0:8000/stocks/
- Status:** 200 OK
- Time:** 17 ms
- Size:** 679 B
- Body (JSON):**

```
{
  "pk": 1,
  "company_name": "new",
  "price": "100000.00",
  "is_growing": true,
  "date_modified": "2023-11-27T18:17:51.945268Z"
},
{
  "pk": 2,
  "company_name": "string",
  "price": "100.00",
  "is_growing": false,
  "date_modified": "2023-11-27T18:17:46.921614Z"
},
{
  "pk": 3,
  "company_name": "gzi-gzi",
  "price": "10000.00",
  "is_growing": false,
  "date_modified": "2023-11-14T12:56:14.982388Z"
}
```

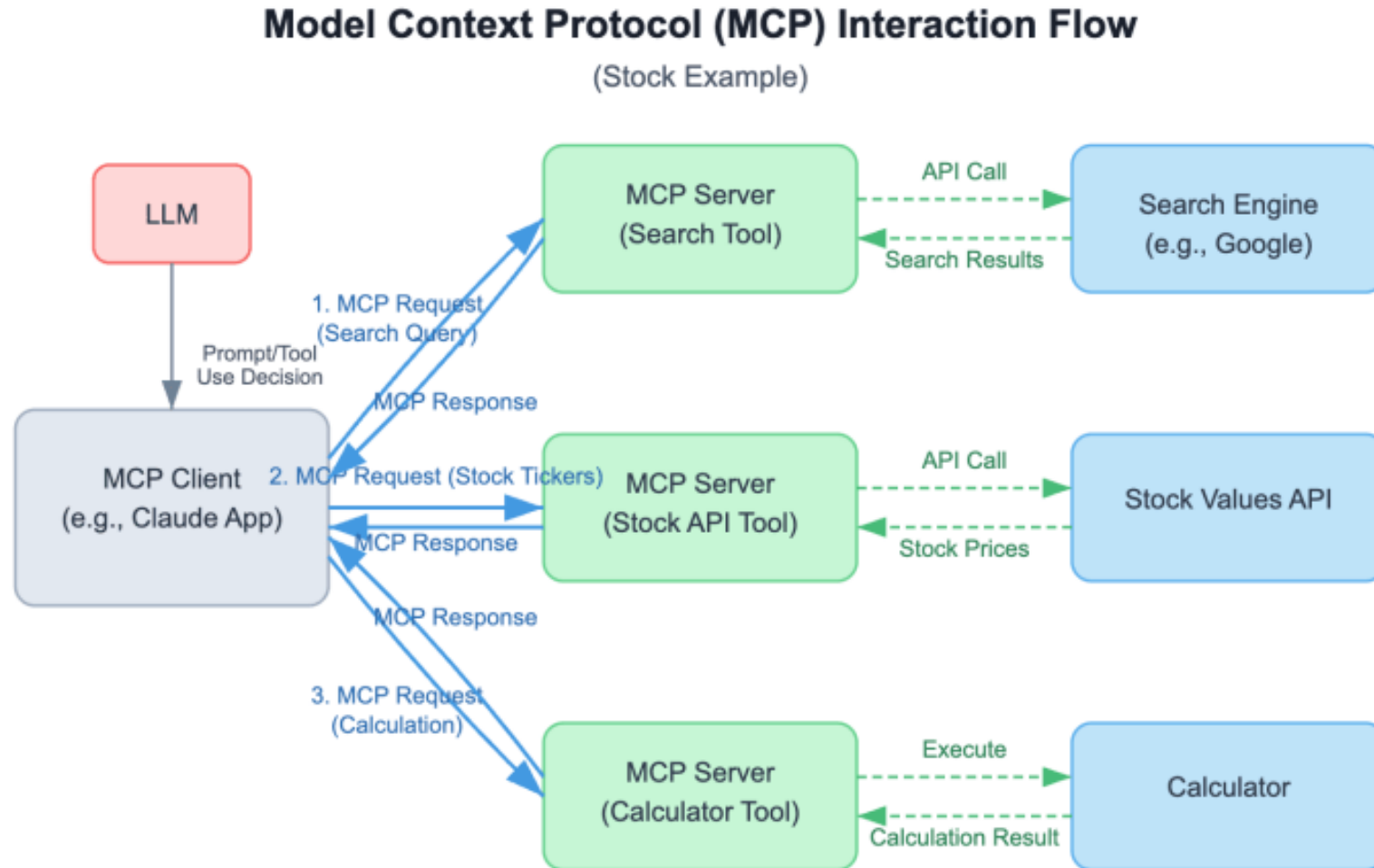
Right Screenshot (POST Request):

- Method:** POST
- URL:** http://0.0.0.0:3000/
- Status:** 200 OK
- Time:** 5 ms
- Size:** 286 B
- Body (JSON):**

```
{
  "pk": 2
}
```

Model Context Protocol (MCP) *

- В ноябре 2024 компания Anthropic представила протокол MCP
- Идея базируется на function calling когда ИИ модель может обращаться к внешним системам
- MCP серверы: **инструменты** (генерация PDF, калькуляторы и тд) и **ресурсы** (доступ к БД, Google Drive и тд)
- **Список** MCP серверов <https://mcpservers.org>



Асинхронное взаимодействие *

В качестве **достоинств** можно выделить

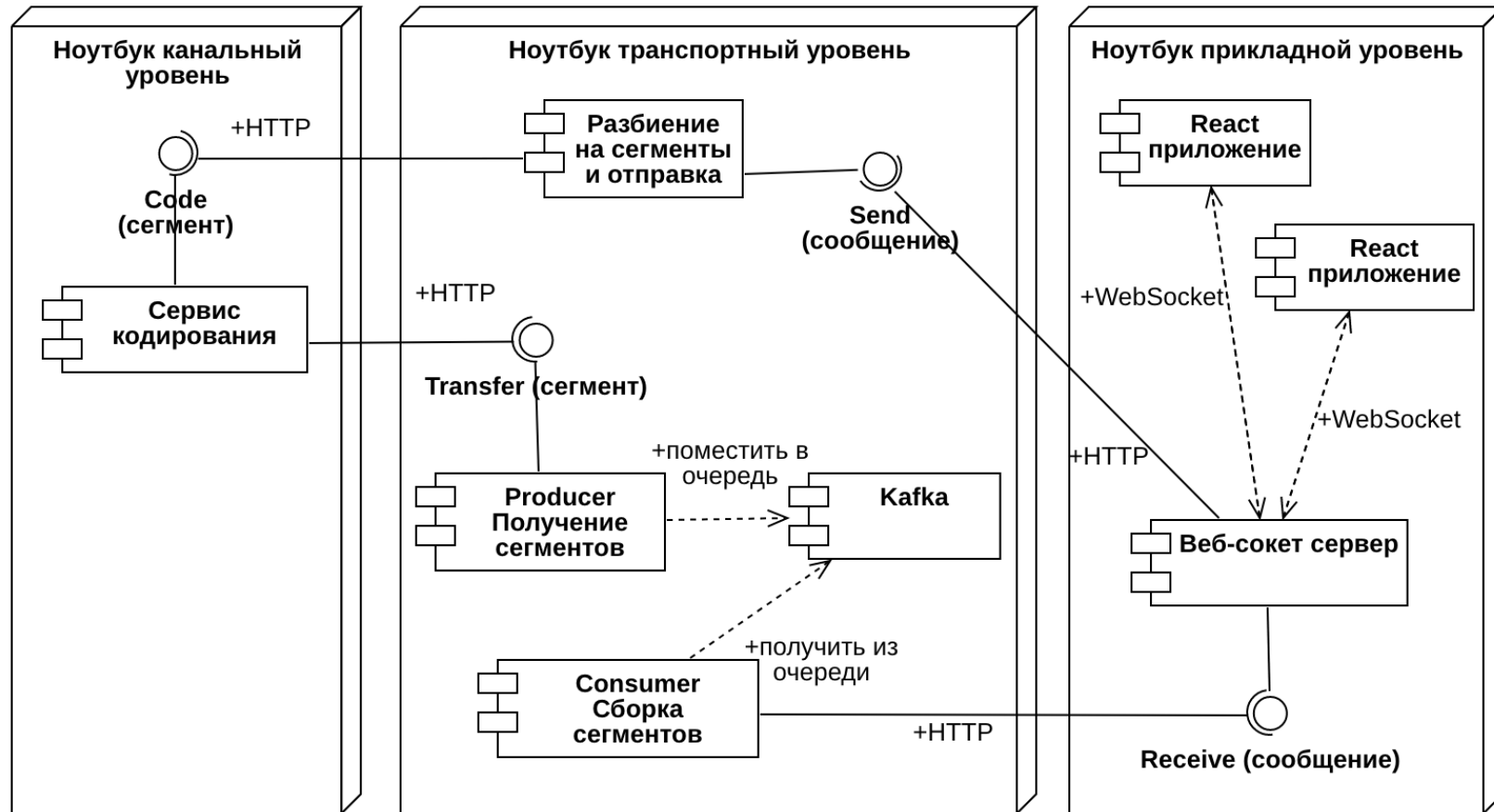
- хорошее использование вычислительных ресурсов серверов (никто не простаивает)
- избегание длительных задержек у пользователя
- Как **недостаток**
- сложность в разработке, особенно когда таких вызовов становится много и они начинают влиять друг на друга
- пиковые нагрузки на сервера
- необходимость добавления real-time web для фронтенда для получения изменений в данных

Решение проблем асинхронности *

- Использование WebSocket, Long Polling, Short Polling и других технологий на фронтенде для обновления данных
- Выделение отдельных сервисов-обработчиков (деление на микросервисы) под каждое выполняемое действие-преобразование данных для сокращения зависимостей
- Использование брокеров и очередей для избегания пиковых нагрузок

Распределенная система обмена сообщениями

- Курсовая Работа
- Необходимо разработать систему обмена текстовыми сообщениями/файлами в реальном времени
- Состоит из трех уровней: прикладной, транспортный и канальный.
- Каждый из уровней реализуется отдельным веб-сервисом.



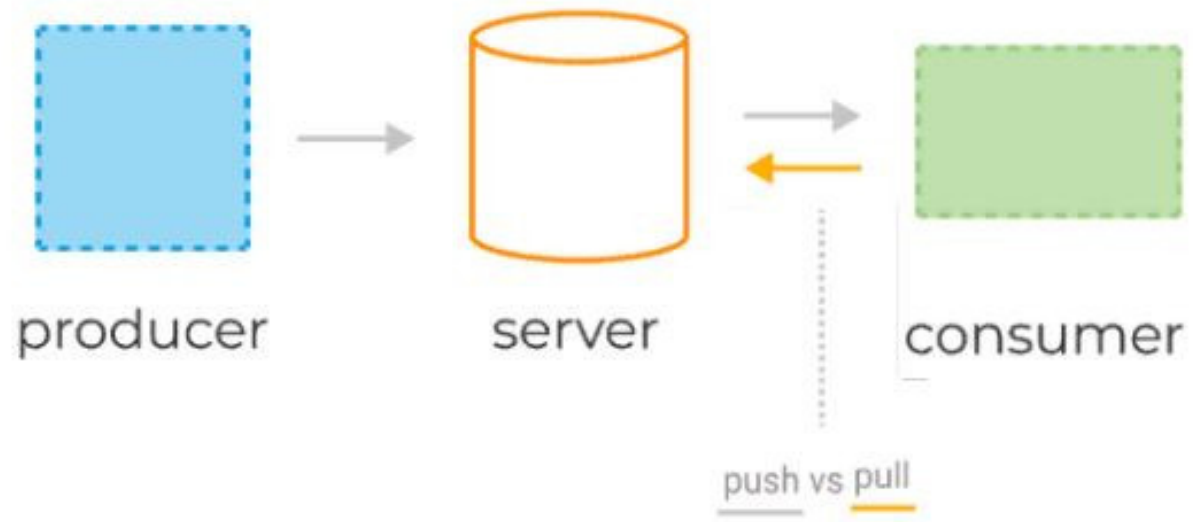
Шлюз сообщений (КР)

- Текстовые сообщения или файлы разбиваются на сегменты по X байт и отправляются отдельными сегментами на канальный уровень.
- При получении сегменты помещаются в очередь, раз в N секунд собираются в единое сообщение и передаются на прикладной уровень. Если часть из сегментов не была принята (1-3 цикла не было новых сегментов по этому сообщению), то на прикладной уровень итоговое сообщение передается с признаком ошибки.
- Каждый пакет-сегмент представляет собой полезную нагрузку, времени отправки (как id сообщения), общую длину сообщения (количество сегментов), номер данного сегмента в сообщении.
- Пример реализации брокера Kafka на Golang по шагам:

https://github.com/iu5git/Networking/tree/main/golang_kafka

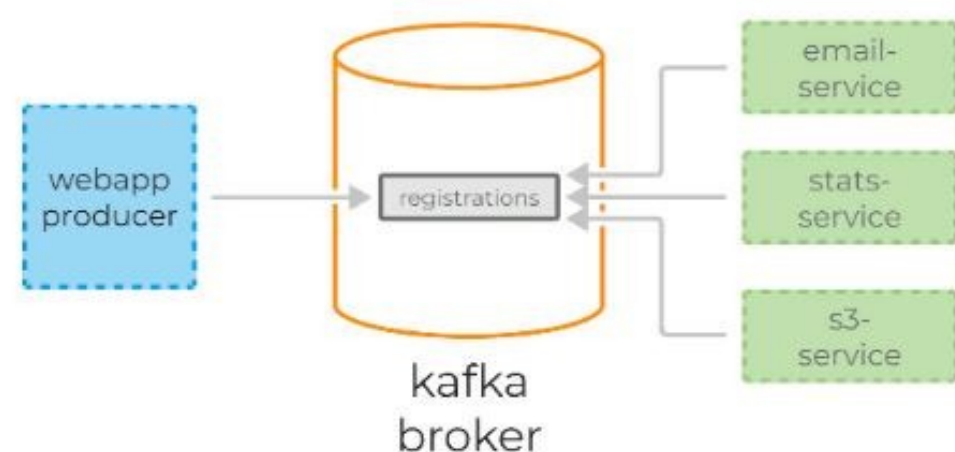
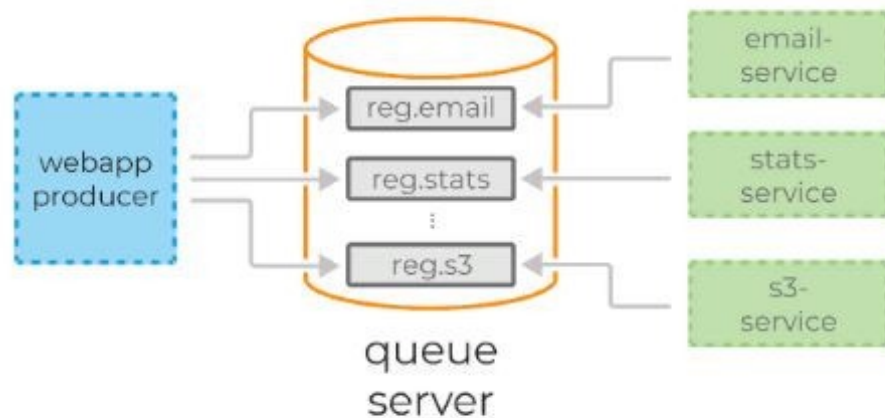
Варианты хранения в брокере

- Redis – резидентная база данных
- Apache Kafka – брокер сообщений
- RabbitMQ – брокер сообщений
- В веб-приложениях очереди часто используются для отложенной обработки событий или в качестве временного буфера между другими сервисами, тем самым защищая их от всплесков нагрузки.



Apache Kafka *

- Представим, что есть некий сайт, на котором происходит регистрация пользователя. Для каждой регистрации мы должны:
 - 1) отправить письмо пользователю,
 - 2) пересчитать дневную статистику регистраций.
- Kafka упрощает задачу - достаточно послать сообщения всего один раз, а консьюмеры сервиса отправки сообщений и консьюмеры статистики сами считают его по мере необходимости



Как реализовать взаимодействие

- Сервисы могут обращаться друг к другу просто по HTTP — хорошо для простых решений (в **нашем курсе**). А можно использовать gRPC
- **gRPC** (Remote Procedure Calls) — это система удалённого вызова процедур (RPC) с открытым исходным кодом, первоначально разработанная в Google
- В качестве транспорта используется HTTP/2, в качестве языка описания интерфейса — Protocol Buffers.
- gRPC предоставляет такие функции как аутентификация, двунаправленная потоковая передача и управление потоком, блокирующие или неблокирующие привязки, а также отмена и тайм-ауты.

Пример gRPC Python

- Описываем структуру данных, сериализатор Protobuf

```
class UserProtoSerializer(proto_serializers.ModelProtoSerializer):  
    class Meta:  
        model = User  
        proto_class = account_pb2.User  
        fields = ['id', 'username', 'email', 'groups']
```

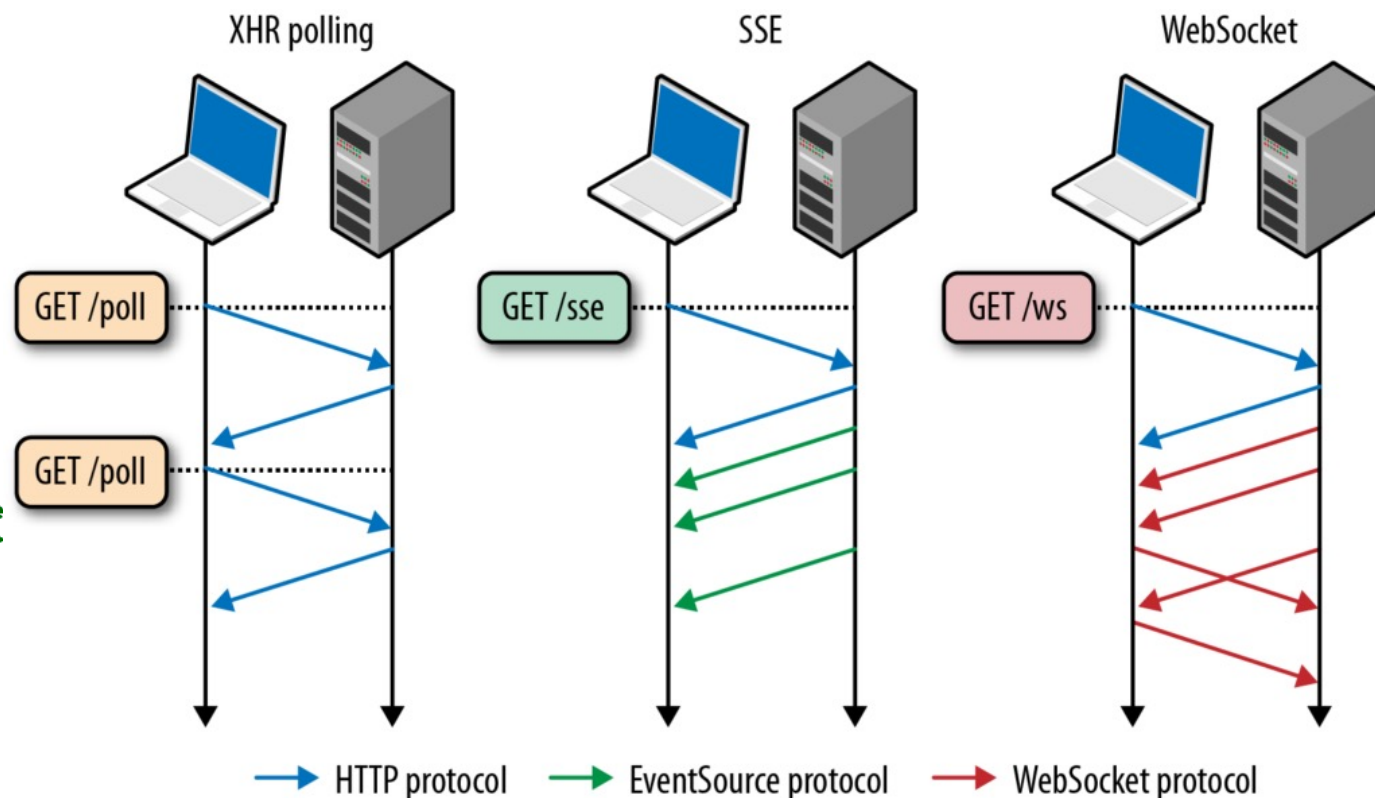
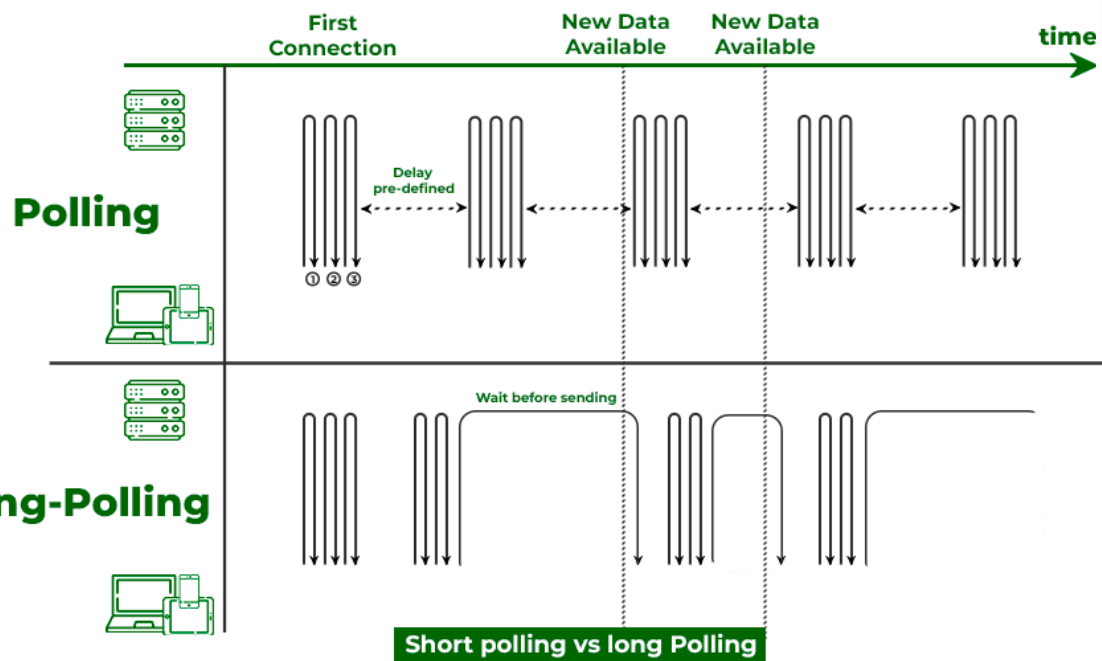
```
class UserService(generics.ModelService):  
    """  
    gRPC service that allows users to be retrieved or updated.  
    """  
    queryset = User.objects.all().order_by('-date_joined')  
    serializer_class = UserProtoSerializer
```

<https://djangogrpcframework.readthedocs.io/en/latest/quickstart.html>

```
syntax = "proto3";  
  
package account;  
  
import "google/protobuf/empty.proto";  
  
service UserController {  
    rpc List(UserListRequest) returns (stream User) {}  
    rpc Create(User) returns (User) {}  
    rpc Retrieve(UserRetrieveRequest) returns (User) {}  
    rpc Update(User) returns (User) {}  
    rpc Destroy(User) returns (google.protobuf.Empty) {}  
}  
  
message User {  
    int32 id = 1;  
    string username = 2;  
    string email = 3;  
    repeated int32 groups = 4;  
}  
  
message UserListRequest {  
}  
  
message UserRetrieveRequest {  
    int32 id = 1;  
}
```

Варианты real-time для фронтенда *

- **Short Polling** – самый простой вариант, просто запросы по таймеру
- При **long polling** сервер задерживает у себя запрос до возникновения события



- EventSource позволяет получить несколько ответов от сервера в разные моменты времени вместо одного
- WebSocket – полноценный двунаправленный обмен с сервером

Реализация Polling *

- Есть разные реализации short polling
- Одна из самых простых – сделать рекурсивный таймер, по истечении которого мы будем выполнять запрос и новый таймер

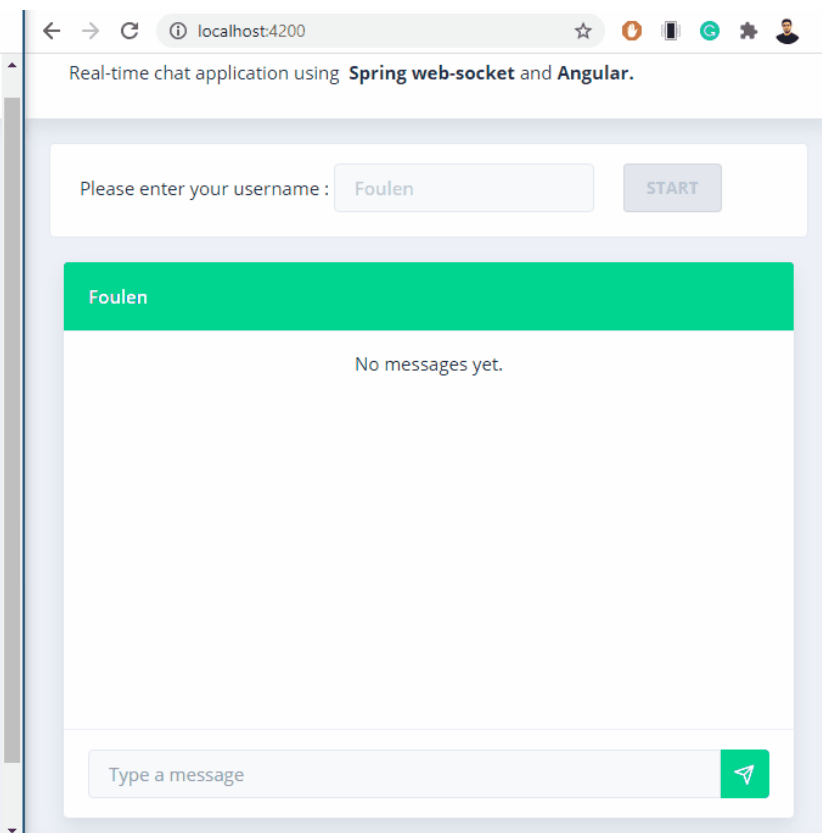
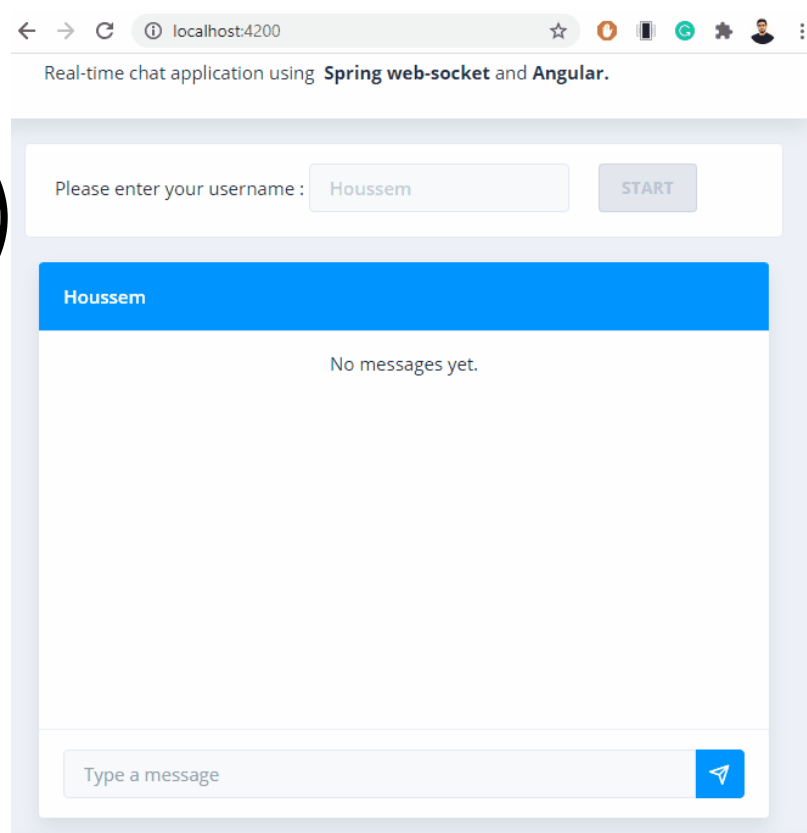
```
let apiTimeout = setTimeout(fetchAPIData, 1000);

function fetchAPIData(){
  fetch('API_END_POINT')
    .then(res => {
      if(res.statusCode == 200){
        // Process the response and update the view.
        // Recreate a setTimeout API call which will be
        apiTimeout = setTimeout(fetchAPIData, 1000);
      }else{
        clearTimeout(apiTimeout);
        // Failure case. If required, alert the user.
      }
    })
    .fail(function(){
      clearTimeout(apiTimeout);
      // Failure case. If required, alert the user.
    });
}
```

WebSocket (КР)

- Пример реализации WebSocket на JS по шагам:

<https://github.com/iu5git/Networking/tree/main/web-socket-chat>

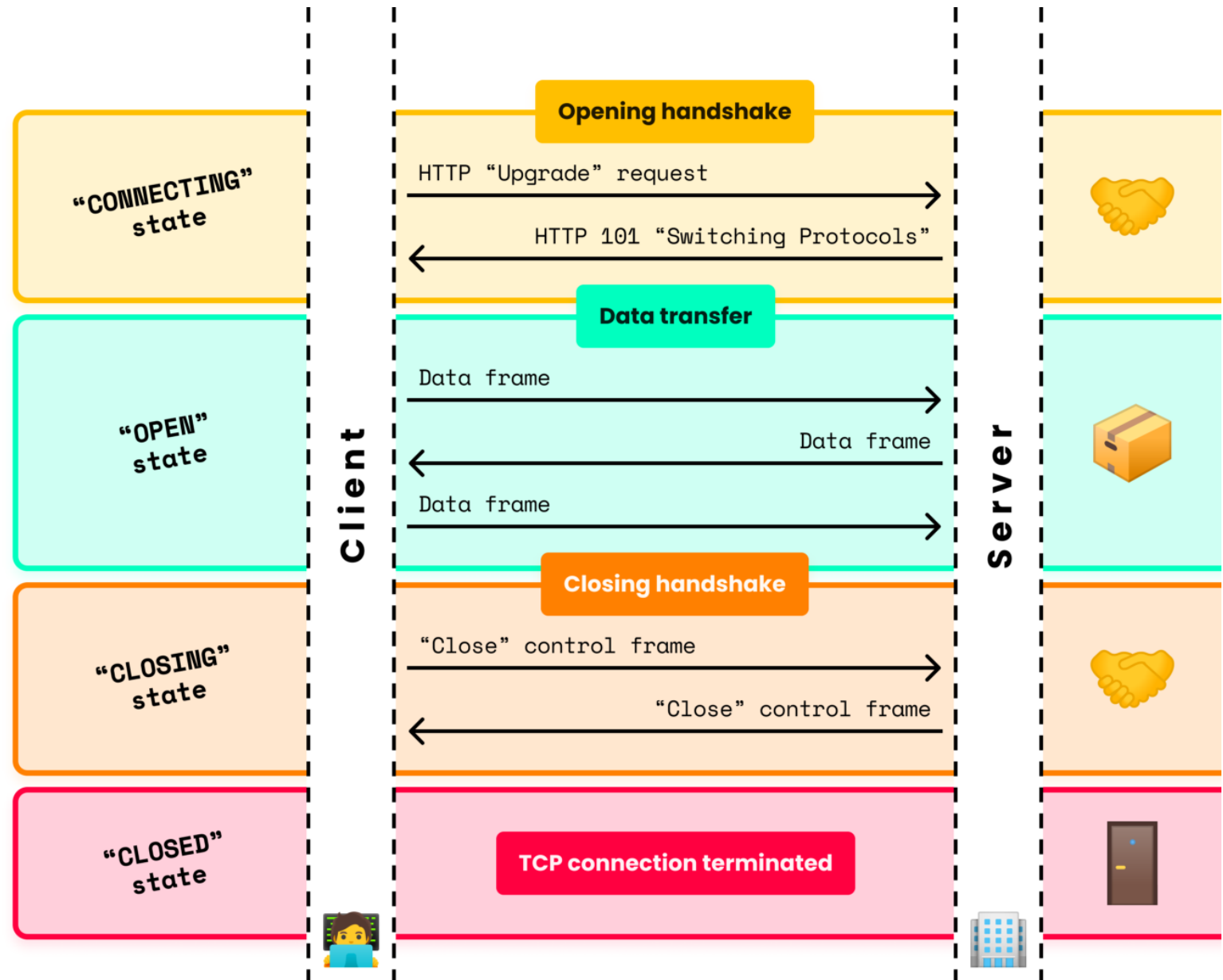


Обмен статусами собеседников через WebSocket

- Необходимо реализовать механизм обмена сообщениями между двумя собеседниками по протоколу WebSocket.
- Должна быть предусмотрена гарантированная отправка и синхронизация версий, если какое-то сообщение пришло раньше-позже (вследствие задержек).
- То есть отображать нужно версии в порядке очередности.

WebSocket *

- Протокол связи поверх TCP-соединения, предназначенный для обмена сообщениями между браузером и веб-сервером, используя постоянное соединение.



WebSocket

- Upgrade HTTP на WebSocket:

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Origin: http://example.com
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 7
```

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
Sec-WebSocket-Protocol: chat
```

- Клиентский скрипт:

```
<html>
  <head>
    <script>
      const websocket = new WebSocket('ws://localhost/echo');

      websocket.onopen = event => {
        alert('onopen');
        websocket.send("Hello Web Socket!");
      };

      websocket.onmessage = event => {
        alert('onmessage, ' + event.data);
      };

      websocket.onclose = event => {
        alert('onclose');
      };
    </script>
  </head>
  <body>
  </body>
</html>
```